

AWS Service Catalog Detailed (DB, Messaging, Deployments, Scaling, Metrics)

This file is a service-by-service catalog for interview revision.

1) Databases and Storage Services

1.1 RDS (Postgres/MySQL/etc.)

Use cases:

- OLTP relational workloads.

Key features:

- backups, Multi-AZ, read replicas, monitoring.

Pitfalls:

- connection storms.
- long transactions and lock contention.

Interview trade-off:

- operational simplicity vs less low-level tuning than self-hosted DB.

1.2 Aurora (MySQL/Postgres compatible)

Use cases:

- high availability relational workloads requiring managed failover.

Key features:

- decoupled storage, fast failover, reader endpoint.

Pitfalls:

- cost visibility and compatibility nuances.

1.3 DynamoDB

Use cases:

- key-value/document, massive scale, low-latency access.

Design rules:

- model access patterns first.
- avoid hot partitions.
- choose partition key carefully.

Features:

- GSIs, TTL, streams, transactions.

1.4 ElastiCache (Redis/Memcached)

Use cases:

- caching, sessions, leaderboards, transient state.

Key decisions:

- cache strategy and invalidation model.
- memory policy and shard sizing.

1.5 S3

Use cases:

- object storage, backups, data lake, static assets.

Must know:

- lifecycle policies.
- storage classes.
- versioning and encryption.

2) Messaging and Event Services

2.1 SQS

Use cases:

- decoupled asynchronous tasks.

Core details:

- Standard (high throughput, at-least-once).
- FIFO (ordering + dedup, lower throughput).

Operational knobs:

- visibility timeout.
- redrive policy to DLQ.
- long polling.

2.2 SNS

Use cases:

- fanout notifications to multiple consumers.

Features:

- subscriptions to SQS, Lambda, HTTP endpoints.
- filter policies for selective routing.

2.3 EventBridge

Use cases:

- event bus integration across services and SaaS.

Features:

- rules-based routing.
- schema registry support.
- scheduled rules.

2.4 Amazon MSK

Use cases:

- managed Kafka streaming.

Key points:

- still need topic/partition/consumer strategy.
- monitor lag, ISR, broker capacity.

3) Deployment Services

3.1 ECS

Use cases:

- containerized microservices with simpler operations than raw k8s.

Modes:

- Fargate (serverless nodes).
- EC2 launch type.

3.2 EKS

Use cases:

- Kubernetes platform with advanced orchestration and portability needs.

Must know:

- node groups.
- control plane managed by AWS.
- IAM roles for service accounts.

3.3 CodePipeline + CodeBuild + CodeDeploy

Use cases:

- managed CI/CD pipelines.

Patterns:

- blue/green and rolling deployments.
- deployment validation hooks.

3.4 Lambda

Use cases:

- event-driven stateless functions.

Limits and concerns:

- runtime limits, cold starts, package size, concurrency controls.

4) Scaling Services and Controls

4.1 Auto Scaling Group (ASG)

- scales EC2 fleet by policies.
- target tracking, step scaling, scheduled scaling.

4.2 ECS Service Auto Scaling

- based on CPU/memory/custom metrics.

4.3 EKS Scaling Stack

- HPA for pods.
- Cluster Autoscaler/Karpenter for nodes.

4.4 Queue-Based Scaling

- scale consumers using queue depth and lag metrics.

5) Metrics, Logging, and Tracing

5.1 CloudWatch Metrics and Alarms

Common backend alarms:

- 5xx rate and latency percentile.
- queue age and DLQ depth.
- DB CPU/connections/storage.
- consumer lag and throughput.

5.2 CloudWatch Logs

- centralized logs with retention policies and insights queries.

5.3 X-Ray or OpenTelemetry

- distributed tracing across API -> DB -> queue workers.

5.4 CloudTrail

- audit trail of API actions for compliance and forensics.

6) Security Services (Critical for Senior Interviews)

6.1 IAM

- users/roles/policies.
- least privilege and role assumption.

6.2 KMS

- key management and envelope encryption.

6.3 Secrets Manager and SSM Parameter Store

- secure secret/config storage and rotation.

6.4 WAF and Shield

- web attack protection and DDoS mitigation for internet-facing apps.

7) Networking Services

7.1 VPC

- subnet design, route tables, internet gateway, NAT gateway.

7.2 ALB/NLB

- L7 routing vs L4 high performance.

7.3 Route 53

- DNS records, health checks, failover routing.

7.4 CloudFront

- CDN and edge caching for lower latency and origin protection.

8) Common Senior-Level AWS Scenarios

Scenario A: Queue backlog incident

- detect with age-of-oldest and inflight metrics.
- scale consumers.
- identify poison messages and DLQ rates.
- tune visibility timeout and retry policy.

Scenario B: DB connection saturation

- enable pooling/proxy.
- reduce app max open conns.
- query optimization and timeout tightening.

Scenario C: Deployment-caused error spike

- canary rollback using alarm gates.
- compare baseline metrics.
- isolate config drift and artifact diff.

Scenario D: Multi-region readiness review

- define RTO/RPO.
- validate replication and failover scripts.
- run game-day drills.

9) Interview Q and A by Service Category

1. Why choose EventBridge over SNS?
 - richer event routing patterns and event bus semantics.
2. When to use DynamoDB over Postgres?
 - predictable key-access high scale with non-relational query requirements.
3. Why combine SQS with Lambda?
 - event-driven processing with managed scaling and decoupled retry behavior.
4. What is one hidden risk in autoscaling?
 - scaling lag and dependency bottlenecks can still breach SLOs.
5. How do you design measurable rollback criteria?
 - predefine latency/error/saturation thresholds and automate comparison windows.

10) Executive Summary Script (60 Seconds)

"In AWS, I design for reliability first: private networking, least-privilege IAM, clear timeout and retry budgets, and observability with SLO-based alarms. For data, I choose RDS/Aurora or DynamoDB based on access patterns and consistency needs. For async, I use SQS/SNS/EventBridge or MSK depending on replay and throughput requirements. Deployment uses canary or blue/green with automated rollback gates. I treat DR as practiced capability with explicit RTO/RPO and regular game-day validation."

11) Service-by-Service Snippet Appendix

11.1 DynamoDB Conditional Write for Idempotency

```
aws dynamodb put-item \  
  --table-name payments_idempotency \  
  --item '{"id":{"S":"req-123"},"status":{"S":"done"}}' \  
  --condition-expression "attribute_not_exists(id)"
```

Interview explanation:

- Prevents duplicate processing under retries.

11.2 EventBridge Rule for Domain Event Routing

```
{  
  "source": ["orders.service"],  
  "detail-type": ["OrderCreated"]  
}
```

Interview explanation:

- Event pattern keeps producers and consumers loosely coupled.

11.3 ALB Health Check Configuration Concept

Typical setup:

- Path: /health
- Success codes: 200-399
- Interval: 15 seconds
- Unhealthy threshold: 2

Interview explanation:

- Faster failure detection must be balanced to avoid false positives.

11.4 S3 Lifecycle Rule Example

```
{  
  "Rules": [  
    {  
      "ID": "logs-transition",  
      "Status": "Enabled",
```

```

    "Filter": {"Prefix": "logs/"},
    "Transitions": [{"Days": 30, "StorageClass": "STANDARD_IA"}],
    "Expiration": {"Days": 365}
  }
]
}

```

Interview explanation:

- Common cost optimization and retention-governance control.

12) L4 AWS Service Selection Drill

12.1 Compute Choice Heuristic

- Lambda: bursty, event-driven, short-lived workloads.
- ECS/Fargate: containerized services without full cluster ops.
- EKS: platform standardization and Kubernetes-native requirements.
- EC2: special performance/runtime control requirements.

12.2 Data Plane Selection Heuristic

- DynamoDB for massive key-value scale and predictable low-latency access.
- Aurora/RDS for relational constraints and complex joins.
- S3 for object durability and cheap large-scale storage.
- ElastiCache for latency-sensitive hot data.

12.3 Messaging Decision Heuristic

- SQS for queue decoupling and retry buffering.
- SNS for fan-out notification.
- EventBridge for event routing with SaaS/service integrations.
- Kinesis/Kafka for high-volume streaming pipelines.

12.4 L4 Grilling Questions

1. Why did you reject serverless for this workload?
2. What is your regional DR strategy and tested RTO/RPO?
3. How do you limit blast radius for IAM misconfiguration?
4. How do you prove architecture cost correctness at scale?